SOFTWARE ENGINEERING · CHAPTER 4

Requirements Engineering

Eliciting, analyzing, specifying, and validating what a system must do

COURSE ROADMAP

Chapter 1 Introduction to Software Engineering

Chapter 2 The Software Process

Chapter 3 Introduction to Agile Methodologies

Chapter 4 *Requirements Engineering (this chapter)*

Learning Objectives

- Explain why RE happens upfront in Waterfall, vs. continuously in Agile.
- Describe the 5 RE phases and the deliverable each one produces.
- Distinguish User Requirements from System Requirements — and avoid the “rabbit hole” trap.
- Write unambiguous requirements using the 5 EARS patterns, and write measurable Non-Functional Requirements.
- Classify requirements with FURPS+ and detect conflicts with an interaction grid.
- Structure a Software Requirements Specification (SRS), including use cases, tables, and UML.
- Apply the 5 validation checks and multiple validation techniques, individually and in combination.
- Explain traceability (RTM) and the Change Control Board (CCB) process.
- Apply all of the above to a running case study: a University Timetabling System.

4.1 WHERE THIS CHAPTER FITS

From Agile Back to Waterfall

In Chapter 3, we studied Agile: iterative development, flexible scope, continuous feedback, requirements that evolve sprint by sprint.

Chapter 3 was the *only* chapter using Agile. From here on — Requirements Engineering, Design, Implementation, Testing — we operate inside the traditional **Waterfall Model.**

Watch for this: the heavy documentation and formal sign-offs in this chapter exist *because* of Waterfall's plan-driven nature — not because they're universally "the right way" to do RE.

The Waterfall Model

A sequential, plan-driven process. Each phase must be completed, reviewed, and formally approved before the next one begins.

1. Requirements
You are here

›

2. System Design

›

3. Implementation

›

4. Testing

No built-in reverse gear. Once a phase is signed off, there is no formal mechanism to revisit it — which is exactly why the Requirements phase you're about to learn is so rigorous: mistakes made here are the most expensive to fix later.

Recap: What Makes Waterfall, Waterfall?

You met the Waterfall model back in Chapter 2 (The Software Process). Three traits define it — and explain almost everything in this chapter:

TRAIT 1

Sequential & Phase-Gated

Work flows in one direction, phase by phase. Each phase ends with a formal review and sign-off — a “gate” — before the next phase may begin.

TRAIT 2

Plan-Driven & Documented

Upfront planning replaces continuous conversation. Formal documents (like the SRS) are the primary way information crosses from one phase — and one team — to the next.

TRAIT 3

Rigid & Predictable

Scope, budget, and schedule are fixed early, which makes the project easy to track — but expensive to change once a phase is baselined.

What Happens in Each Phase?

Seeing the whole pipeline helps you understand why Requirements carries so much weight: everything downstream is built directly on top of it.

Phase	What Happens	Key Deliverable
1. Requirements (this chapter)	Discover, analyze, specify, and validate what the system must do and the constraints it must satisfy.	Baselined SRS
2. System Design	Translate the SRS into software architecture, data models, and interface designs.	Design Specification
3. Implementation	Developers write and unit-test code that satisfies the design — and, ultimately, the original requirements.	Working Modules
4. Testing	Integrate all modules and verify / validate the complete system against the original requirements.	Verified System

Waterfall: Advantages vs. Disadvantages

ADVANTAGES

- Clear milestones and a strong audit trail — easy for stakeholders to track progress.
- Predictable budget and schedule, fixed early in the project.
- Well suited to fixed-scope, safety-critical, or heavily regulated projects.
- Formal documentation eases handoff between teams and over staff turnover.

DISADVANTAGES

- Little room for change once a phase is baselined — scope changes are slow and costly.
- Stakeholders don't see working software until very late in the project.
- Requirements errors found during Testing are the most expensive to fix of all.
- High risk of building the wrong thing if early requirements are misunderstood.

This is why RE matters so much: *the biggest disadvantage above — costly late-stage fixes — is exactly what a rigorous Requirements phase exists to prevent.*

Waterfall RE vs. Agile RE

Dimension	Waterfall Requirements Engineering	Agile Requirements Engineering
Timing & Cadence	Completed entirely upfront, in one dedicated phase.	Continuous, in short iterations across the project.
Documentation	Heavy, formal Software Requirements Specification (SRS).	Lightweight backlog items, user stories, acceptance criteria.
Scope Change	Rigid — requires a formal change-control procedure.	Flexible — backlog is continuously reprioritized.
Sign-off Target	Stakeholders sign off before architecture begins.	Refined continuously with product owner, sprint by sprint.

4.2

The Requirements Engineering Roadmap

Five interlocking phases, refined through a spiral of increasing depth

What Is Requirements Engineering?

Requirements define *what* a system should do, plus the constraints it must operate under. **Requirements Engineering (RE)** is the process of discovering, analyzing, documenting, and validating those needs — five interlocking phases, each with its own deliverable (Sections 4.3–4.7):



Not a rigid pipeline: Validation regularly sends work back to Elicitation or Analysis when gaps are found, and Specification draws directly on what Analysis has organized.

Three Loops of Progressive Refinement

The internal RE process is iterative: the same activities repeat at three increasing depths.

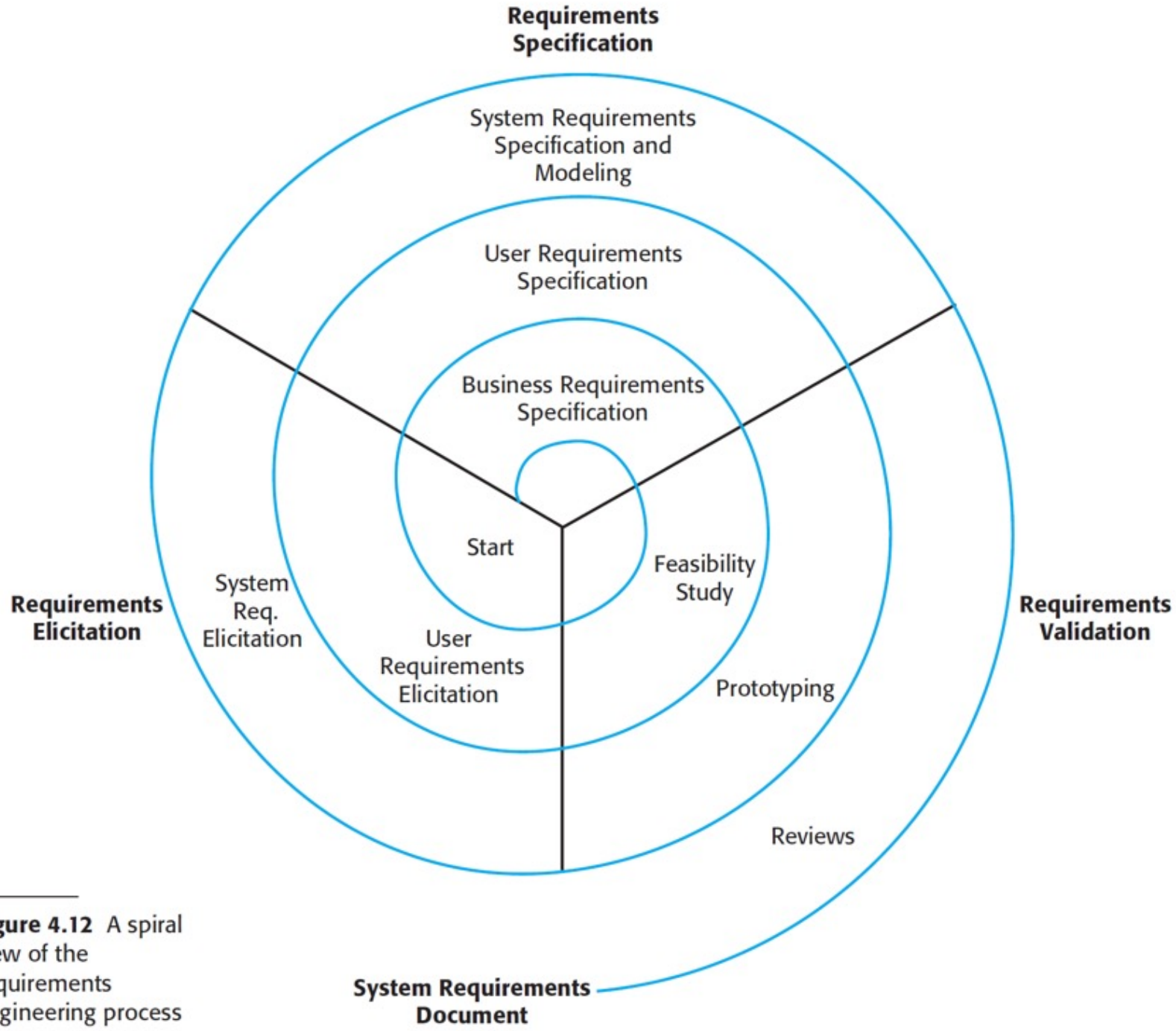


Figure 4.12 A spiral view of the requirements engineering process

Why the Direction Matters

Business → User → System.

Each loop's validated output becomes the input the next loop refines. Skipping straight to the System loop without validating Business and User needs first is one of the most common — and costly — mistakes in Waterfall RE.

4.3

Phase 0: Feasibility Study

Is this project even worth pursuing?

Core Objectives & Report Outcomes

CORE OBJECTIVES

- Does the system contribute to overall organizational objectives?
- Can it be engineered with current technology, within budget and schedule?
- Can it integrate with existing infrastructure and legacy systems?

Key inputs: executive summaries, business mandates, user expectations, legacy documentation, market benchmarks.

THE FEASIBILITY REPORT — 3 OUTCOMES

1. Proceed

Technically & financially viable — move directly to Elicitation.

2. Scope Down

Viable only if specific high-cost / high-risk requirements are removed.

3. Cancel Project

Unfeasible, unviable, or misaligned with corporate goals.

CASE STUDY CHECKPOINT

Feasibility: Timetabling System — Proceed

A university's IT Steering Committee commissions a Feasibility Study for a proposed **Timetabling System.** Current technology (a standard web/mobile stack) can meet the need, the budget covers a small dev team for two semesters, and the system can integrate with the existing student records database.

Outcome: Proceed. The Feasibility Report clears the project to move into Phase 1 — Elicitation, where we meet the people who will actually use the system.

4.4

Phase 1: Requirements Elicitation

Gathering raw stakeholder needs — without judging or organizing them yet

Where Do Requirements Come From?

SOURCE 1

Operational Managers

Workflow throughput, resource allocation, auditing, staff performance oversight.

SOURCE 2

Corporate Policies

Business constraints, financial rules, brand guidelines, org hierarchy.

SOURCE 3

Physical Environment

Hardware limits, ambient conditions, locations, network access, sensor limits.

SOURCE 4

Privacy Laws & Regulators

Statutory compliance — e.g., GDPR, HIPAA, PCI-DSS.

Sources in Practice: Concrete Examples

OPERATIONAL MANAGERS

“The system shall generate a weekly summary report detailing total room allocation requests, approval turnaround times, and unutilized room slots grouped by department.”

CORPORATE POLICIES

“All purchases or resource allocations exceeding $5,000 must undergo a multi-level approval sequence with digital signatures from the Dept. Head and Financial Controller.”

PHYSICAL ENVIRONMENT

“The handheld barcode scanner must stay readable in direct sunlight up to 100,000 lux and operate continuously between -20°C and 50°C.”

PRIVACY LAW (GDPR)

“The system shall automatically purge personal identification records, activity logs, and emails exactly 30 days after account termination (GDPR Art. 17).”

Elicitation Techniques & How to Document Them

Different sources call for different techniques — and each technique has a natural way to document what it uncovers.

Technique	What It Captures	How to Document It
Interviews	Deep, individual insight from a specific stakeholder.	Interview notes attributed to the stakeholder, with date — kept close to their own wording.
Workshops / JAD Sessions	Consensus and trade-offs across multiple stakeholders in one room.	Workshop minutes, plus a running requirements log of every raw statement raised.
Observation & Shadowing	Tacit knowledge — habits users perform automatically and forget to mention.	Field notes and simple “as-is” workflow diagrams of what was observed.
Questionnaires / Surveys	Broad input from a large stakeholder population.	Tabulated survey results, summarized by question.
Document Analysis	Existing policies, forms, and legacy manuals that already encode a rule.	Extracted requirement statements, each citing its source document.
Prototyping (Low-Fidelity)	Reactions to a concrete UI idea, before code is written.	Annotated wireframes plus a log of stakeholder feedback per screen.

The Elicitation Register

Across every technique above, the single most useful artifact is an **Elicitation Register** — a running table that captures each raw statement exactly as it was gathered, before any analysis:

Field	Purpose
Informal ID	A temporary tag (e.g., E-014) so the statement can be referenced before it earns a formal UR/SR ID in Phase 2 or 3.
Raw Statement	The stakeholder's need, captured as close to their own words as possible.
Source	Who said it, and through which technique (interview, workshop, observation, document, survey).
Date	When it was captured — useful when requirements evolve or contradict each other over time.
Status	Draft, by default. (Only Phase 2 promotes an entry to “Organized” or “Classified.”)

Example — Elicitation Register Excerpt

ID	Raw Statement	Source	Date	Status
E-001	“I want to see my class schedule on my phone, not just on the desktop portal.”	Sarah (Student), interview	3 Feb	Draft
E-002	“When I need to swap a room urgently, right now I have to email the admin and hope she sees it in time.”	Dr. Li (Lecturer), workshop	5 Feb	Draft
E-003	“Room swaps should never bypass my sign-off — I'm accountable for room conflicts.”	Ms. Lee (Facility Administrator), interview	6 Feb	Draft

These three entries recur throughout this chapter: **E-001** and **E-002** are organized into User Requirements in Phase 2, while **E-003** becomes a business constraint attached to E-002's requirement, rather than a standalone entry.

Use Case Diagrams: A First Map of Actors and Use Cases

Before writing a detailed use case description, it helps to see the whole system at a glance — a lightweight sketch of who does what.

Use Case Diagram. Groups system functions visually by user role ("Actors") and by system boundary. It requires no analysis yet — it is simply a map of the raw material, not a judgment about it.

Each oval — each candidate use case — becomes a target for a full **Use Case Description** next. The diagram scopes *which* interactions exist; the description spells out *how* one plays out, step by step.

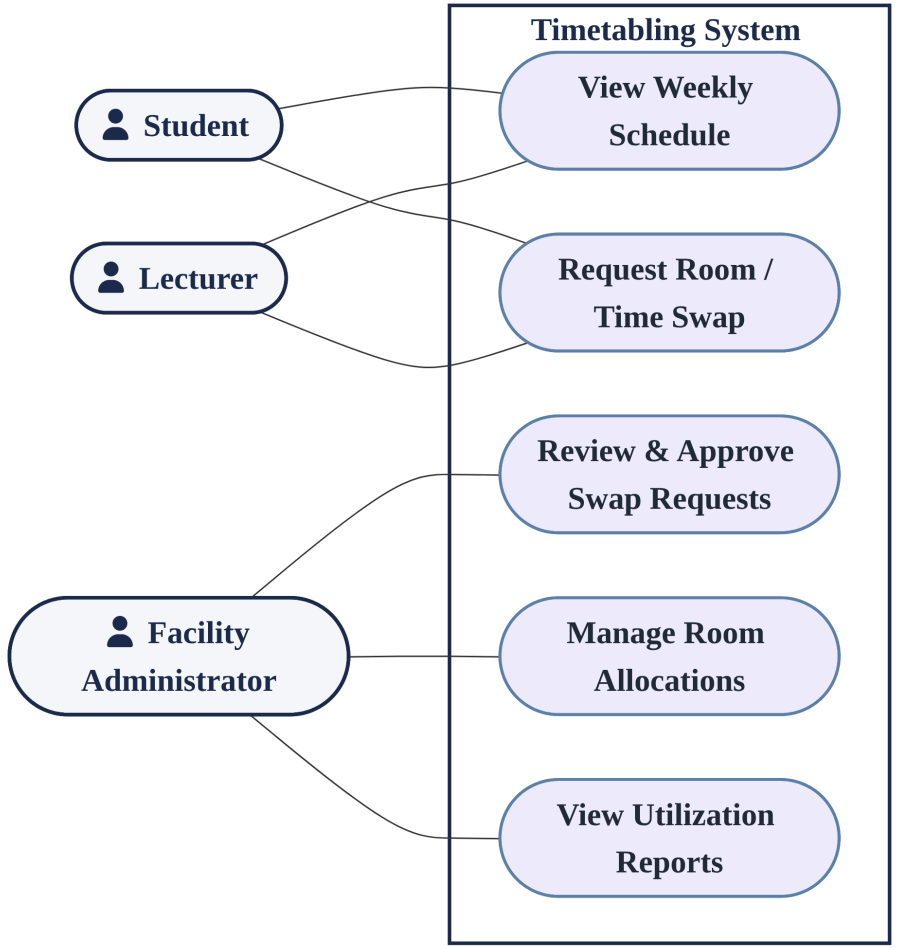


Fig. 4.1 — High-level use case diagram, Timetabling System (broad view: actors and system boundary only)

The Use Case Description Template

Bridges high-level user goals and concrete system execution — captures the main success scenario, variations, exceptions, and state boundaries.

Use Case ID & Name

Unique identifier and descriptive action name.

Primary Actor

The specific user role initiating the action.

Pre-conditions

System state that MUST be true before execution.

Post-conditions

State of the system after successful execution.

Main Success Scenario

Step-by-step numbered interaction flow (the “happy path”).

Extensions / Alt. Flows

Branching paths handling optional choices or errors.

UC-04: Request Urgent Timetable Room Swap

Field	Description
Primary Actor	University Lecturer
Pre-conditions	1. Lecturer is authenticated. 2. Lecturer has an assigned session within 48 hours.
Post-conditions	1. Swap request created: PENDING_ADMIN_REVIEW. 2. Notification queued for the Facility Administrator. 3. Room slot flagged SWAP_REQUESTED.

UC-04: Main Success Scenario & Extensions

MAIN SUCCESS SCENARIO

- Lecturer opens “My Schedule” and selects a class session.
- System displays class details and available alternative rooms.
- Lecturer selects a target room and enters a mandatory reason.
- Lecturer submits the request.
- System validates room capacity and equipment against class size.
- System logs the request and returns a tracking ID (REQ-XXXX).

EXTENSIONS (ALTERNATE FLOWS)

a. Target room capacity < enrollment:

1. System warns: “Selected room capacity (30) is smaller than enrollment (45).”
2. System prompts Lecturer to pick another room or request an override.

b. Target room concurrently requested:

1. System flags the concurrency conflict.
2. System rejects submission and refreshes the room list.

4.4.3 FREQUENTLY ASKED

Should Use Case Descriptions Be Written in Stage 1 or Stage 3?

Both — but for different purposes, at different levels of finish:

Stage 1 (Elicitation): drafted informally, in the stakeholder's own language, focused on the happy path. Its job is to confirm the team correctly understood one specific interaction.

Stage 3 (Specification): the same use case is revisited, finalized, and formalized — every extension and alternate flow is fully specified, and it becomes part of the baselined SRS (Section 4.6.1).

Common Challenges & Resolution Strategies

Challenge	Resolution Strategy
1. “Tacit Knowledge” — users forget to explain core steps.	Process Ethnography & Shadowing — observe users in real time.
2. Domain Jargon — internal acronyms & terminology.	Domain Glossary — shared dictionary, signed off by all teams.
3. Unrealistic “Wish List” needs.	Feasibility & Cost-Value Matrix — attach rough cost estimates.
4. Changing Political Dynamics.	Multi-Perspective Group Workshops — facilitated consensus sessions.

Output of the Elicitation Phase

Elicitation ends when the team has captured a broad, honest picture of stakeholder needs — not a tidy one. Nothing below is yet classified, prioritized, or checked for conflicts.

Deliverable	Description
Elicitation Register	The running log of raw, informally-tagged requirement statements.
Draft Use Case Diagram	A lightweight actor/system-boundary sketch scoping which interactions exist — not yet analyzed.
Draft Use Case Descriptions	Happy-path-only narratives confirming the team's understanding of key interactions.
Stakeholder & Viewpoint List	Who was consulted, and which Viewpoint they represent (formalized as VOA in Phase 2, Section 4.5.2).
Domain Glossary (Draft)	A first pass at shared terminology, to be refined as Analysis proceeds.
Supporting Artifacts	Interview notes, workshop minutes, field observations, low-fidelity prototypes.

CASE STUDY CHECKPOINT

Elicitation: Three Stakeholders, Three Raw Statements

The team runs interviews and a workshop with Sarah (a Student), Dr. Li (a Lecturer), and Ms. Lee (the Facility Administrator). The Elicitation Register fills up:

E-001 (Sarah): “I want to see my class schedule on my phone, not just on the desktop portal.”
E-002 (Dr. Li): “When I need to swap a room urgently, right now I have to email the admin and hope she sees it in time.”
E-003 (Ms. Lee): “Room swaps should never bypass my sign-off — I'm accountable for room conflicts.”

These raw, unorganized entries — plus a draft Use Case (UC-04, from E-002) — move to Phase 2 for analysis.

4.5

Phase 2: Requirements Analysis & Classification

Turning a raw pile of requests into a structured framework

From Raw Statements to a Structured Foundation

Requirements Analysis & Classification takes the raw, unfiltered Elicitation Register and turns it into a structured, unambiguous foundation for Specification. It has two jobs — and neither one is capturing new facts, which is Elicitation's job:

1. Organize

Sort each raw statement into User Requirements (what stakeholders need) vs. System Requirements (how it will be built), and check every viewpoint is represented (Section 4.5.1, VOA).

2. Classify

Rewrite each statement in EARS syntax (Section 4.5.2) so it says one thing only, tag it under FURPS+ (Section 4.5.3), and check it against every other requirement for hidden conflicts (Section 4.5.4).

Input → Output. In: the raw Elicitation Register (Section 4.4.3). Out: an organized, EARS-formatted, conflict-checked requirement set — ready to be written into the formal SRS in Phase 3 (Specification).

User Requirements vs. System Requirements

	User Requirements	System Requirements
Written for	Stakeholders (non-technical)	Developers and testers
Style	Natural language, simple tables, diagrams	Structured, technical, precise
Answers	What services does the user need? What are the business goals?	What exact functions, API endpoints, data structures, and constraints must exist?

Why this distinction matters. User Requirements keep the SRS in language every stakeholder can validate and sign off on; System Requirements give developers and testers the precision to build and verify. Blur the two and either stakeholders can't confirm you understood them, or developers are left guessing — which is exactly how the “rabbit hole” trap (next slide) happens.

Avoid the “Rabbit Hole” Trap

Novice engineers often dive into low-level System Requirements — database schemas, API protocols, caching — before fully understanding the high-level User Requirements. This builds a well-engineered system that solves the *wrong* problem.

Rule of thumb: *you cannot specify how a system should work until you fully understand what problem the user is trying to solve.*

Viewpoint-Oriented Analysis (VOA)

Purpose: VOA groups requirements by the perspective they come from, so analysis works through each perspective in turn instead of one undifferentiated pile:

INTERACTOR

Direct Users

People who directly interact with the UI.

INDIRECT

Indirect Stakeholders

Never touch the system, but rely on its outputs (auditors, executives).

DOMAIN

Domain Constraints

Environmental and technical limits imposed by the domain itself.

Why we need it. Requirements gathered from only one viewpoint — usually Direct Users — systematically miss Indirect Stakeholders' needs and Domain Constraints, exactly the kind of gap that surfaces expensively during Validation, or worse, after deployment. Checking all three viewpoints before Specification is what makes the analysis complete, not just plausible.

EARS: Easy Approach to Requirements Syntax

Before the raw Elicitation Register can be organized, classified, or checked for conflicts, every entry needs one fixed sentence template — so two readers can no longer interpret the same requirement two different ways. Five patterns cover every case:

1	**Ubiquitous**	`The <system> shall <response>.`	*Always true*
2	**Event-Driven**	`WHEN <trigger>, the <system> shall <response>.`	*A specific event fires it*
3	**State-Driven**	`WHILE <in state>, the <system> shall <response>.`	*Only during a state*
4	**Option-Driven**	`WHERE <feature included>, the <system> shall <response>.`	*Optional feature/config*
5	**Unwanted Behavior**	`IF <error>, THEN the <system> shall <response>.`	*Error handling*

Pattern 1 — Ubiquitous

Always true — no trigger, no condition

```
The <system> shall <system response>.
```

- The system shall display all date and time values in UTC format.
- The system shall log all user authentication attempts in an immutable audit table.
- The application shall display the company logo in the top-left header of all UI screens.

Patterns 2 & 3 — Event-Driven and State-Driven

Event-Driven

```
WHEN <trigger>, the <system>
shall <response>.
```

- WHEN a lecturer submits a room swap request, the system shall email the facility administrator within 5 seconds.
- WHEN a user clicks “Export CSV,” the system shall generate and download a sanitized report file.

State-Driven

```
WHILE <in state>, the <system>
shall <response>.
```

- WHILE database sync is in progress, the system shall disable the manual data import button.
- WHILE the network connection is offline, the system shall queue pending transactions locally.

Patterns 4 & 5 — Option-Driven and Unwanted Behavior

Option-Driven

```
WHERE <feature included>, the <system> shall <response>.
```

- WHERE biometric hardware is present, the app shall allow fingerprint authentication.
- WHERE dark mode is selected, the app shall render the UI in the high-contrast dark palette.

Unwanted Behavior

```
IF <error/trigger>, THEN the <system> shall <response>.
```

- IF an invalid password is entered 3 times, THEN the system shall lock the account for 15 minutes.
- IF an uploaded file exceeds 25 MB, THEN the system shall reject it with a size-limit error.

EARS Quick Reference

Pattern	Keyword	Template
Ubiquitous	(none)	The <system> shall <response>.
Event-Driven	WHEN	WHEN <trigger>, the <system> shall <response>.
State-Driven	WHILE	WHILE <in state>, the <system> shall <response>.
Option-Driven	WHERE	WHERE <feature included>, the <system> shall <response>.
Unwanted Behavior	IF ... THEN	IF <error/trigger>, THEN the <system> shall <response>.

Why this comes before Organizing and Classifying. A requirement in EARS form has its trigger, condition, and response as separate, identifiable parts — exactly what's needed to tag it under FURPS+ and to compare it against others in a conflict-detection grid.

Functional Requirements, by Category

Functional Requirements (FR) define a specific behavior the system must perform. They answer *"what does the system do?"* and are written using the EARS patterns you learned in Section 4.5.2.

Category	Example Functional Requirement
Data Management	WHEN a student updates their profile address, the system shall update the central student information database.
User Management	The system shall allow an administrator to deactivate a lecturer account.
Reporting	The system shall generate a weekly room-utilization report grouped by department.
Integration	WHEN a new semester is created, the system shall synchronize course codes with the university's ERP system.

Writing Measurable Non-Functional Requirements

Non-Functional Requirements (NFR) define a quality attribute — not what the system does, but how well. An NFR is only useful if a tester can look at it and say Pass or Fail with no argument.

```
The <system> shall <behavior> within/under <threshold + unit> when/under <condition>.
```

Metric

What is being measured (time, throughput, error rate, uptime %, concurrent users).

Threshold

A specific number with a unit. “Fast” is not a threshold; “2 seconds” is.

Condition

The context under which the threshold applies (load level, network type, hardware).

From Vague to Measurable

Vague (Unmeasurable)	Rewritten (Measurable)
“The system shall be fast.”	“The system shall return search results within 2 seconds for 95% of queries under normal load (≤100 concurrent users).”
“The system shall be user-friendly.”	“A first-time user shall complete the room-swap workflow in under 3 minutes without external help.”
“The system shall be secure.”	“The system shall encrypt all data at rest using AES-256 and require multi-factor authentication for admins.”
“The system shall be reliable.”	“The system shall maintain 99.9% uptime measured monthly, excluding scheduled maintenance.”

Resolution — the FURPS+ Framework

Stakeholders rarely separate the two on their own. Engineers are expected to identify FR apart from NFR. — **FURPS+** gives six categories to sort them into:

F — Functionality
Features, capabilities, security

U — Usability
Human factors, aesthetics, docs

R — Reliability
Availability, accuracy, recoverability

P — Performance
Speed, throughput, response time

S — Supportability
Testability, maintainability

+ — Constraints
Design, Implementation, Interface, Physical

Example: *"The system shall print invoices instantly" splits into a Functional req. (support printing) + a Performance req. (completes within 2 seconds).*

4.5.3 FREQUENTLY ASKED

Is the “F” in FURPS+ the Same as a Functional Requirement (FR)?

No —It is not a true Functional Requirement.

”F” in FURPS+ (Functionality) is a category — feature set, security scope, audit and compliance controls — used to group and review related requirements together. It is a label, not a testable sentence.

FR (Functional Requirement) is one individual, EARS-formatted, testable statement (Section 4.6.3)

Why put an F in an NFR-style framework? They describe a quality of a function, not the function itself. But got grouped under "F" because they don't fit neatly under Usability, Reliability, Performance, or Supportability either.

THE SIMPLE TEST

One Question Separates Them

Can I remove this requirement and the system still does the same task, just less safely, traceably, or controllably?

NO — THE TASK DISAPPEARS

→ It's a true FR

Write it as an EARS statement and attach it directly to the feature's acceptance criteria.

Example: *"The system shall allow a user to cancel a pending order before execution."*

YES — THE TASK STILL WORKS

→ It sits under "F"

Document it once as a cross-cutting review item, referenced by every feature it touches.

Example: *"All trading actions require MFA" or "every trade generates an immutable audit log entry."*

SIDE-BY-SIDE · TRADING SYSTEM EXAMPLES

FR or “F”? A Working Example

Requirement	Type	Why
The system shall allow a user to place a limit order specifying price and quantity.	**FR**	Removing it deletes a feature.
The system shall allow a user to cancel a pending order before execution.	**FR**	A discrete, testable action.
The system shall encrypt all API traffic using TLS 1.2+.	**“F”**	Order placement still works — just insecurely — without it.
The system shall log every trade with timestamp, user ID, and order details.	**“F”**	Adds traceability; doesn’t change what the trade does.

Rule of thumb: *if the rule is core business logic tied to one feature, write it as an FR inside that feature. If it’s a cross-cutting policy applied across many features, document it once under “F.”*

Resolution — Matrix-Based Interaction Analysis

An architectural conflict occurs when satisfying one requirement makes another harder to satisfy (e.g., real-time speed vs. heavy encryption). Score every pair: **+1 Compatible, 0 Neutral, −1 Conflict.**

	FR1: Mobile	NFR1: Load < 3s	NFR2: Encryption	FR2: Swap
FR1: Mobile	—	+1	0	0
NFR1: Load < 3s	+1	—	**−1 (Conflict)**	0
NFR2: Encryption	0	**−1 (Conflict)**	—	0
FR2: Swap	0	0	0	—

Reading it: NFR1 and NFR2 conflict — encryption overhead works against the load-time budget, an explicit trade-off is needed before any code is written. Note: tags like “NFR1” are shorthand for the full EARS statement written in Section 4.5.1 — the grid only works because each requirement is already precise enough to compare.

4.6

Phase 3: Requirements Specification

Translating analyzed needs into a formal, unambiguous SRS

From Organized Requirements to a Baselined SRS

Requirements Specification takes the organized, classified requirements from Phase 2 and writes them into one authoritative, contractually-binding document: the **Software Requirements Specification (SRS).** Once the Joint Steering Committee signs it, the SRS becomes the baseline every later phase is measured against.

From User to System

Every User Requirement (UR) is refined into one or more precise System Requirements (SR) — see the worked UR → SR examples ahead.

One Document, Every Artifact

EARS-formatted FRs, measurable NFRs, use cases, and UML diagrams are all assembled into the standard SRS structure, next.

Governed, Not Casual

A dedicated sign-off process and two recurring failure modes — ambiguity and over-specification — are guarded against before baselining.

Standard SRS Structure

Every artifact this chapter has built — requirements, EARS statements, use cases, the interaction grid — is ultimately assembled into one document: the Software Requirements Specification (SRS).

Section	Contents
1. Introduction	Purpose, scope, definitions/glossary, references, document overview.
2. Overall Description	Product perspective, product functions summary, user characteristics, constraints.
3. Specific Requirements	3.1 FR (EARS syntax) · 3.2 External Interfaces · 3.3 NFR (FURPS+, measurable) · 3.4 Use Cases & UML.
4. Appendices	Data dictionary, Requirements Traceability Matrix, interaction analysis grid, revision history.
5. Approval / Sign-off	Signatures from the Joint Steering Committee, marking the document baselined.

Governance & Key Challenges

WHO SIGNS OFF? — THE JOINT STEERING COMMITTEE

Role	Responsibility
Lead Architect & Business Analyst	Verifies technical feasibility and conceptual completeness.
Client Representative / Sponsor	Confirms requirements represent business objectives.
Project Manager	Confirms scope matches contractual budget and timeline.

TWO KEY CHALLENGES

Natural Language Ambiguity

“Requirement Smells” — words like “fast” or “secure” mean different things to different readers. EARS syntax (Section 4.5.1) prevents this.

Over-Specification

Accidentally prescribing implementation details (e.g., a specific SQL table) instead of external behavior.

CASE STUDY CHECKPOINT

Specification: From User Requirement to System Requirement

PHASE 1 — ELICITATION (USER LEVEL)

UR-1

“The system shall allow students to view their weekly timetable on mobile devices.”

Drafted from Sarah's raw statement (E-001). Focuses strictly on operational need — no database schema, no network protocol.

PHASE 3 — SPECIFICATION (SYSTEM LEVEL)

SR-1.1

“WHEN a student logs into the mobile app, the system shall fetch and render the current week's schedule in calendar format within 3.0 seconds over a 4G connection.”

Derived after UR-1 was organized in Phase 2 and approved by the Steering Committee — gives developers and QA concrete, testable parameters.

CASE STUDY CHECKPOINT

Specification: The Second Chain — UR-2 to SR-2.1

PHASE 1 — ELICITATION (USER LEVEL)

UR-2

“The system shall allow lecturers to request an urgent swap to another available room for a scheduled class session.”

Drafted from Dr. Li's raw statement (E-002). E-003 (Ms. Lee's sign-off condition) attaches here as a business constraint, not a separate UR.

PHASE 3 — SPECIFICATION (SYSTEM LEVEL)

SR-2.1

“IF the requested room's capacity is less than the class's enrollment, THEN the system shall reject the swap and prompt the lecturer to select another room.”

Derived after UR-2 was organized in Phase 2 — formalizes the exact capacity check already previewed as UC-04's extension 5a.

CASE STUDY CHECKPOINT

Specification: The Draft FR1–FR5 / NFR1 Table

ID	EARS Type	Requirement
FR1	Ubiquitous	The system shall display weekly class schedules in a calendar view.
FR2	Event-Driven	WHEN a room change is submitted, the system shall alert the administrator.
FR3	State-Driven	WHILE database sync is in progress, the system shall show a loading indicator.
FR4	Option-Driven	WHERE text-to-speech is enabled, the system shall read out room details.
FR5	Unwanted Behavior	IF an invalid university ID is entered, THEN the system shall show an error.
NFR1	Ubiquitous	The mobile app shall load timetables within 3.0 seconds over 4G.

4.7

Phase 4: Requirements Validation

Do the specified requirements really reflect what stakeholders expect?

The Five Validation Checks

1

Validity

Do features align with real operational needs?

2

Consistency

Do any requirements contradict each other?

3

Completeness

Were any scenarios simply forgotten?

4

Realism

Is this physically or financially achievable?

5

Verifiability

Can this be objectively tested (Pass/Fail)?

The Five Checks — Concrete Examples

Check	Concrete Example
Validity	Confirm room swaps really require admin sign-off per university policy.
Consistency	Catch that FR-2 says “all requests auto-approved” while NFR-4 says “manual confirmation required.”
Completeness	Notice the spec never defines account recovery when a user loses their 2FA device.
Realism	Prove 50,000 requests/sec is unachievable on a $10,000 cloud budget.
Verifiability	Rewrite “System shall render UI quickly” into “home screen renders in under 1.5s” — a clear Pass/Fail test.

Requirements Validation Techniques

The five checks describe what to look for. Techniques describe how a team actually goes looking for it — and are rarely used in isolation.

Technique	How It Works	Best At
Reviews / Inspections	A panel walks through the SRS against a checklist, formally (Fagan) or informally.	Consistency, Completeness, Validity
Prototyping	Build a throwaway or evolutionary prototype and have stakeholders interact with it.	Validity, Completeness (UI-heavy)
Test-Case Generation	For each requirement, attempt to write a concrete Pass/Fail test case.	Verifiability
Walkthroughs	An informal, step-by-step presentation of requirements to stakeholders.	Validity, Completeness
Model / Consistency Checking	Cross-check System Models (diagrams, state charts) against each other and the text.	Consistency
Traceability Analysis	Use the RTM to find orphan requirements or orphan code.	Completeness, Consistency

Applying the Techniques to the Case Study

TEST-CASE GENERATION

SR-2.1

“IF capacity < enrollment, THEN reject the swap” converts directly into Given/When/Then Test Case TC-UNIT-402.

REVIEW

Catching an Unmeasurable NFR

“The mobile app shall load timetables quickly” fails the Measurability Checklist — sent back rewritten as NFR1.

PROTOTYPING

Confirming UC-04

A low-fidelity mockup shown to Dr. Li reveals a missing mandatory “reason for swap” field — a Completeness gap.

WALKTHROUGH

Confirming Validity with Ms. Lee

The finalized FR1–FR5 / NFR1 table is presented to Ms. Lee, who confirms FR2 matches her expectations.

CASE STUDY CHECKPOINT

Validation: The SRS Is Ready for Baselining

Running the techniques above over the Timetabling System's draft SRS finds:

- **No remaining Verifiability failures** — every FR/NFR can be tested.
- **No Consistency conflicts** beyond the already-logged NFR1/NFR2 trade-off, to be resolved during System Design.
- **One Completeness gap** — the missing “reason for swap” field, now added to UC-04.

With these closed, the SRS is ready for baselining — see the RTM next, in Section 4.8.

4.8

Requirements Traceability & Change Control

Protecting the baselined SRS from unmanaged scope creep

Forward, Backward, and Bidirectional

Type	Direction	Purpose
Bidirectional	Both ways	Connects user requirements down to specs, code, and tests — and back.
Forward	Requirement → Code/Test	Every requirement is designed, coded, tested. Prevents “omitted scope.”
Backward	Code/Test → Requirement	Every piece of code maps to an authorized requirement. Prevents “Gold Plating.”

What is “Gold Plating”? Building unrequested features or unnecessary complexity — “while I'm in here, I'll also add…” It inflates cost and schedule without adding contracted value. Backward traceability is the main defense: if code doesn't map to a requirement ID, it shouldn't exist.

The Requirements Traceability Matrix (RTM)

Req ID	Description	Level	Design Module	Test Case	Status
UR-1	View weekly schedule on mobile.	User	TimetableUI	TC-UI-101	Approved
SR-1.1	WHEN tab opened, fetch JSON <2.0s over 4G.	System	ScheduleCtrl	TC-API-204	Passed
UR-2	Lecturer can request urgent room swap.	User	SwapModule	TC-UI-105	Approved
SR-2.1	IF capacity < enrollment, THEN reject swap.	System	CapacityChk	TC-UNIT-402	Passed

Why the RTM Matters

IMPACT ANALYSIS

“What breaks?”

When a change is proposed, engineers consult the RTM to see which modules, schemas, and tests are affected.

GAP ANALYSIS

“What's missing?”

Finds orphan requirements (no design module) or orphan code (no originating requirement).

TEST COVERAGE

“Are we ready?”

QA confirms 100% of requirements have a corresponding test before release.

Why Waterfall Needs Formal Change Control

Real-world conditions change: laws update, priorities shift, hardware limits appear. Uncontrolled changes cause **Scope Creep** — so Waterfall routes every change through the **Change Control Board (CCB).**

THE CCB EVALUATES EVERY CHANGE FROM FOUR ANGLES

Project Sponsor

Business necessity; approves financial cost additions.

Project Manager

Impact on schedule, milestones, staffing.

Lead System Architect

Technical feasibility, refactoring complexity.

QA Manager

Re-testing effort, regression risk.

The Change Request Lifecycle

1. Change Request
Stakeholder identifies a need

›

2. Impact & Cost Analysis
Architect & PM assess impact

›

3. CCB Review
Business / Tech / Finance / PM vote

IF APPROVED

- Update the SRS baseline (e.g., v1.0 → v1.1).
- Update the RTM and code.
- Adjust budget and schedule.
- Re-sign the contract.

IF REJECTED

- Document the rationale in the CCB Log.
- Retain the existing baseline.

Change Control — Step-by-Step Execution

1	**Submit the ECR**	A formal Engineering Change Request logs the request ID, submitter, description, and justification.
2	**Impact Assessment**	Leadership uses the RTM to estimate files to modify, migrations required, work-hours, and milestone impact.
3	**CCB Deliberation**	The board weighs business benefit against cost, schedule delay, and technical risk.
4	**Formal Decision**	Rejected → archived with justification. Approved → new SRS version, updated RTM, revised schedule/budget.

CASE STUDY CHECKPOINT

Traceability & Change Control: A Real ECR

Six months after baselining, a new data-privacy regulation requires all schedule data in transit to be encrypted end-to-end — the risk the Interaction Grid had only flagged as “future” back in Analysis. A Facility IT lead submits an ECR.

The ECR writes the anticipated risk into a real requirement for the first time: **NFR2 — “The system shall encrypt all schedule and swap-request data in transit using TLS 1.3, and at rest using AES-256.”**

The team consults the RTM: **the change touches SR-1.1** and its test case TC-API-204 will need updating, since encryption overhead affects the 3.0s load-time budget — precisely the NFR1-vs-NFR2 trade-off logged during Analysis. The CCB approves it — the SRS is re-baselined as v1.1, and the RTM rows for SR-1.1 and the new NFR2 are updated.

QUICK REFERENCE

Telling the ID Codes Apart

Several ID prefixes appear side by side in this chapter — here's how to tell them apart:

Prefix	Meaning	Assigned During	Example
E-xxx	Raw statement from Elicitation, unanalyzed.	Phase 1 (Elicitation)	E-002
UC-xx	Use Case: an actor–system interaction.	Phase 1 → Phase 3	UC-04
UR-x	User Requirement — organized, for stakeholders.	Phase 2 (Analysis)	UR-2
SR-x.x	System Requirement — technical, from one UR.	Phase 3 (Specification)	SR-2.1
FRx / NFRx	Functional / Non-Functional Requirement (SRS's list).	Phase 3 (Specification)	FR1, NFR1
TC-xxx	Test Case — verifies one requirement (in the RTM).	Phase 4 (Validation)	TC-API-204

Reading the numbering. SR-2.1 means “the 1st System Requirement derived from UR-2” — the decimal ties it to its parent UR. FR/NFR numbers, by contrast, just count sequentially through the SRS's formal requirement list; they don't encode a parent ID.

4.9

Case Study: The Complete Picture

One requirement, followed continuously through every phase of this chapter

Meet the Stakeholders

STUDENT

Sarah

Wants to view her weekly class timetable on her phone.

LECTURER

Dr. Li

Needs to request urgent room swaps for lecture sessions.

FACILITY ADMIN

Ms. Lee

Reviews and approves room allocation and swap requests.

Timetabling System SRS

All three stakeholders' needs feed into one Software Requirements Specification.

Nothing Was Introduced in Isolation

Each phase's checkpoint built directly on the previous one — two parallel chains, both traced start to finish: Sarah's E-001 → UR-1 → SR-1.1, and Dr. Li's E-002 → UR-2 → SR-2.1.

Phase	What Happened in Our Case Study
0. Feasibility	IT Steering Committee approves the project: Proceed.
1. Elicitation	Interviews produce a raw Elicitation Register (E-001, E-002, E-003) and a draft UC-04.
2. Analysis & Classification	E-001 → UR-1 and E-002 → UR-2, both tagged with FURPS+; a future NFR1-vs-encryption conflict is flagged (Section 4.5.4).
3. Specification	UR-1 → SR-1.1 and UR-2 → SR-2.1 are refined into System Requirements; the FR1–FR5 / NFR1 EARS table is drafted for the SRS.
4. Validation	Test-case generation, a review, a prototype walkthrough, and a walkthrough close out remaining gaps.
Traceability & Change Control	The RTM traces both chains end-to-end; a later ECR finally writes NFR2 (encryption) and re-baselines SR-1.1 to v1.1.

KEY TAKEAWAYS

Chapter 4 in Review

- Waterfall RE is upfront, plan-driven — baselined and validated before design begins.
- RE has 5 phases: Feasibility, Elicitation, Analysis & Classification, Specification, Validation.
- Elicitation captures raw needs; Analysis is where organizing and classifying happens.
- User Requirements come first — avoid the “rabbit hole” into System detail.
- EARS syntax (5 patterns) and FURPS+ classification remove ambiguity; every NFR must be measurable.
- The SRS groups FRs by module and NFRs by FURPS+, and includes use cases, tables, and UML.
- 5 Validation Checks describe what to look for; techniques (reviews, prototyping, test cases) describe how.
- The RTM and the CCB protect the baselined SRS from scope creep.